



CYBERILLSEC · A CYBERILL INITIATIVE

SENTRY

Security Monitoring & Cyber Threat Intelligence Platform

CAHIER DES CHARGES MAÎTRE & DOSSIER D'INGÉNIERIE LOGICIELLE

Auteur & Architecte : Godwill FOKA

Rôle : Lead Architect & Security Engineer

Organisation : CYBERILL (Berlin)

Version : v1.0.0 (Master Baseline)

Date : Juillet – Septembre 2026

Statut : Approuvé pour Développement

SENTRY — CAHIER DES CHARGES & DOSSIER D'INGÉNIERIE LOGICIELLE

Plateforme de Surveillance des Menaces et Cyber Threat Intelligence (CyberillSec)

De la Phase d'Avant-Projet aux Spécifications Techniques et au Guide d'Onboarding

PROJET	: SENTRY (Security Monitoring & Threat Intelligence Platform)
INITIATIVE	: CyberillSec — A CYBERILL Initiative
AUTEUR & ARCHITECTE	: Godwill FOKA (Lead Architect & Security Engineer)
DESTINATAIRE	: Équipe d'Ingénierie Logicielle / Développeurs Onboarding
VERSION DU DOCUMENT	: v1.0.0 (Master Engineering Baseline)
DATE DE BASELINE	: Juillet — Septembre 2026
DÉPÔT GITHUB	: github.com/GodwillFoka/sentry (Miroir GitLab: gitlab.com/GodwillFoka/sentry)
CLASSIFICATION	: Interne CYBERILL — Document Fondateur de Référence

SOMMAIRE GÉNÉRAL

1. CADRE MÉTIER & VULGARISATION POUR LE DÉVELOPPEUR NOVICE

- 2. 1.1 Qu'est-ce que CYBERILL et l'écosystème CyberillSec ?
- 3. 1.2 Qu'est-ce que SENTRY et quelle mission remplit-il ?
- 4. 1.3 Le Lexique du Développeur Cyber (Les concepts fondamentaux expliqués sans jargon)
- 5. 1.4 La journée type de l'analyste SOC et comment SENTRY transforme son travail

6. PHASE D'AVANT-PROJET & ANALYSE PRÉALABLE

- 7. 2.1 Contexte et État de l'Art en 2026
- 8. 2.2 Problématique d'Ingénierie et Besoins Utilisateurs
- 9. 2.3 Étude d'Opportunité et Analyse de Faisabilité (Technique, Opérationnelle, Économique)
- 10. 2.4 Matrice des Risques Pré-Projet et Stratégies d'Atténuation
- 11. 2.5 Analyse SWOAT (Strengths, Weaknesses, Opportunities, Aspirations, Threats)

12. CAHIER DES CHARGES FONCTIONNEL (CdCF)

- 13. 3.1 Vue d'ensemble des 6 Modules Métier
- 14. 3.2 Matrice des Exigences Fonctionnelles Globales (RF-01 à RF-30)
- 15. 3.3 Module 1 : Foundation & Noyau d'Authentification / Configuration
- 16. 3.4 Module 2 : Moteur de Collecte & Ingestion Threat Feeds (IOC Engine)
- 17. 3.5 Module 3 : Moteur de Veille et Détection de Vulnérabilités (CVE Tracker Engine)
- 18. 3.6 Module 4 : Moteur de Gestion des Incidents & Timeline (Incident Management)
- 19. 3.7 Module 5 : SOC Dashboard & Moteur d'Agrégation Analytique
- 20. 3.8 Module 6 : Moteur de Détection & Threat Hunting (Rules Engine)

21. 3.9 Cas d'Usage Détaillés (Diagrammes de Flux & Scénarios Nominaux/Alternatifs)

22. CAHIER DES CHARGES TECHNIQUE (CdCT) & EXIGENCES NON-FONCTIONNELLES

23. 4.1 Matrice des Exigences Non-Fonctionnelles (RNF : Performance, Sécurité, Résilience)

24. 4.2 Justification Rigoureuse de la Stack Technologique

25. 4.3 Architecture Logicielle : Clean / Layered Architecture

26. 4.4 Modèle Conceptuel et Relationnel de Données (MCD / MLD & Schémas SQL)

27. 4.5 Spécifications des Contrats d'Interface & Endpoints API REST (OpenAPI 3.1)

28. 4.6 Architecture de Déploiement et Conteneurisation (Docker Compose & Services)

29. PLANNING OPÉRATIONNEL & DÉCOUPAGE DES 44 TÂCHES (J1 À J63)

30. 5.1 Macro-Planning, Fenêtres Temporelles et Calendrier de Référence

31. 5.2 Les 6 Jalons Majeurs d'Acceptation (Milestones M1 à M5 + V1.0)

32. 5.3 Décomposition Exhaustive des 44 Tâches Quotidiennes

33. 5.4 Protocole des Bilans Hebdomadaires du Jeudi & Production des Rapports

34. GUIDE D'ONBOARDING & MANUEL PRATIQUE DU DÉVELOPPEUR (DAY-1 SURVIVAL GUIDE)

35. 6.1 Prérequis Système et Installation Pas à Pas de l'Environnement Local

36. 6.2 Structure Physique du Dépôt de Code Expliquée Dossier par Dossier

37. 6.3 Standards d'Ingénierie : Style de Code, Typage Strict, Linting et Tests

38. 6.4 GitFlow, Stratégie de Branches et Règles de Pull Request

39. 6.5 Procédure Pas à Pas pour Réaliser sa Première Tâche

1. CADRE MÉTIER & VULGARISATION POUR LE DÉVELOPPEUR NOVICE

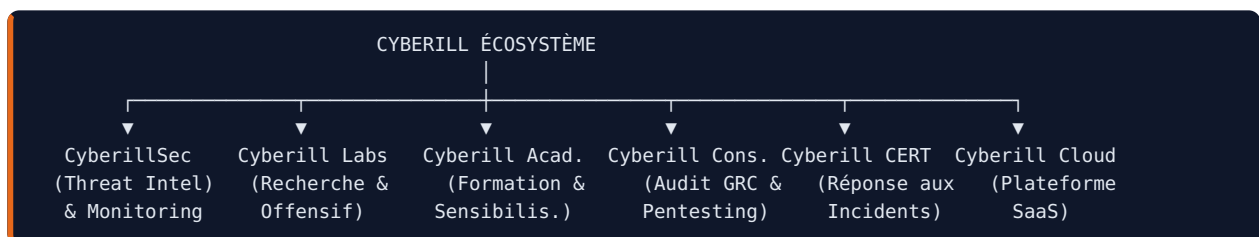
Bienvenue dans l'équipe d'ingénierie logicielle de CYBERILL. Si tu n'as jamais fait de cybersécurité auparavant, **ce chapitre a été écrit spécifiquement pour toi**. Il va te donner les repères indispensables pour comprendre ce que tu codes, pourquoi chaque ligne est critique, et quel problème réel tu résous pour les utilisateurs.

1.1 Qu'est-ce que CYBERILL et l'écosystème CyberillSec ?

CYBERILL est une initiative d'ingénierie et de résilience numérique fondée à Berlin par Godwill FOKA. Sa devise fondatrice est :

"Engineering Cyber Resilience. Empowering Digital Trust."

CYBERILL s'articule autour de 6 piliers majeurs :



CyberillSec est le bras armé de veille stratégique et de Threat Intelligence. Sa mission : collecter, analyser, corréler et vulgariser les données relatives aux cyberattaques mondiales afin que les organisations ne soient plus prises au dépourvu.

1.2 Qu'est-ce que SENTRY et quelle mission remplit-il ?

SENTRY est l'application concrète, le moteur logiciel qui propulse CyberillSec.

Dans l'industrie, un analyste en sécurité passe souvent **60% à 70% de son temps** à copier-coller des indicateurs entre 10 outils différents, à lire des bulletins de sécurité illisibles et à trier des alertes en double.

SENTRY résout ce problème en créant un cerveau centralisé :

1. Il va chercher automatiquement les renseignements sur les menaces sur Internet (flux publics, bases gouvernementales, flux communautaires).
2. Il normalise ces données dans un format unique et propre.
3. Il calcule un score de risque objectif pour chaque vulnérabilité et menace.
4. Il offre une interface unifiée (API REST, CLI en ligne de commande, Dashboard SOC) pour décider en quelques secondes d'une remédiation au lieu de plusieurs heures.

1.3 Le Lexique du Développeur Cyber (Les concepts fondamentaux)

Pour être à l'aise avec le code de SENTRY, voici les termes exacts que tu manipuleras dans les modèles de base de données et les endpoints d'API :

- **CTI (Cyber Threat Intelligence)** : Le renseignement sur les menaces. C'est l'ensemble des données collectées sur les attaquants, leurs motivations, leurs capacités et leurs méthodes.
- **IOC (Indicator of Compromise — Indicateur de Compromission)** : L'empreinte numérique laissée par un attaquant sur un système ou un réseau.
- *Exemples concrets* : Une adresse IP malveillante (`185.220.101.5`), un nom de domaine frauduleux (`secure-login-bank.xyz`), une URL de phishing, ou le hash cryptographique d'un binaire malware (`e3b0c44298fc...`).
- **CVE (Common Vulnerabilities and Exposures)** : L'identifiant universel d'une faille de sécurité dans un logiciel ou matériel (ex: `CVE-2026-16812`). Chaque faille répertoriée dans le monde possède un identifiant unique CVE.
- **CVSS (Common Vulnerability Scoring System)** : Une note technique de 0.0 à 10.0 mesurant la sévérité intrinsèque d'une faille. (Exemple : 9.8 = Critique, exploitable à distance sans mot de passe).
- **EPSS (Exploit Prediction Scoring System)** : Un score probabiliste (entre 0.0 et 1.0) calculé quotidiennement par l'organisme FIRST, qui prédit la probabilité qu'une CVE soit réellement exploitée dans la nature dans les 30 prochains jours. Un score > 0.7 indique une probabilité très élevée d'attaque imminente.
- **CISA KEV (Known Exploited Vulnerabilities)** : Le catalogue gouvernemental américain de référence listant les failles dont l'exploitation active par des pirates sur des cibles réelles a été formellement prouvée. Si une CVE est dans le KEV, le patch est une urgence absolue.
- **TTPs (Tactics, Techniques, and Procedures) & MITRE ATT&CK** : La cartographie mondiale des comportements d'attaquants. Les tactiques représentent le *POURQUOI* (ex: "Accès Initial"), les techniques représentent le *COMMENT* (ex: "T1566 - Phishing").
- **Threat Hunting** : La démarche proactive qui consiste, pour un défenseur, à fouiller dans son réseau pour chercher des traces d'attaques qui auraient contourné les antivirus traditionnels.
- **Règles Sigma & YARA** :
 - *Sigma* : Des règles génériques écrites en YAML pour détecter des comportements suspects dans les journaux d'événements (logs).
 - *YARA* : Des signatures textuelles ou binaires pour identifier des familles de malwares dans des fichiers.

1.4 La journée type de l'analyste SOC et l'apport de SENTRY

Imagine un analyste de sécurité (Alex) qui commence son quart de travail à 08h00 : * **Sans SENTRY** : Il ouvre 15 onglets (CISA, NVD NIST, AlienVault, Twitter, BleepingComputer). Il lit 40 articles. Il voit passer une faille Fortinet. Il doit chercher manuellement son CVSS, vérifier si un exploit existe sur GitHub, vérifier si des adresses IP attaquantes sont connues, puis ouvrir un ticket dans son gestionnaire d'incidents. Il est 11h30, il n'a encore rien bloqué. * **Avec SENTRY** : SENTRY a tourné toute la nuit via ses tâches d'arrière-plan asynchrones. À 08h01, Alex tape `sentry dashboard show` ou consulte `/api/v1/dashboard/summary`. SENTRY lui affiche immédiatement : * Les 3 CVE critiques de la nuit ayant un score de risque > 80 (car KEV = Oui et EPSS > 0.85). * Les 40 nouveaux IOCs découverts, dédoublés et reliés à la campagne en cours. * L'incident créé automatiquement avec sa chronologie pré-remplie. En 5 minutes, Alex peut lancer le blocage firewall et notifier son RSSI. C'est cette valeur que ton code va produire.

2. PHASE D'AVANT-PROJET & ANALYSE PRÉALABLE

Tout projet d'ingénierie logicielle d'envergure nécessite une phase de cadrage amont rigoureuse pour valider sa pertinence, sa faisabilité et ses frontières avant d'engager le moindre effort de développement.

2.1 Contexte et État de l'Art en 2026

L'année 2026 marque un tournant critique dans l'industrie de la cybersécurité : 1. **L'industrialisation des ransomwares (RaaS)** : Les attaquants exploitent des vulnérabilités logicielles moins de 48 heures après leur divulgation publique. Le modèle traditionnel de patching mensuel ("Patch Tuesday") est devenu obsolète. 2. **L'inflation normative européenne (Directive NIS 2 & DORA)** : Les entreprises ont l'obligation légale de maintenir une veille active sur leurs vulnérabilités et de notifier les incidents majeurs sous des délais stricts (24h/72h). 3. **Le coût prohibitif des solutions de Threat Intelligence propriétaires** : Des plateformes comme Recorded Future, Mandiant Advantage ou CrowdStrike Falcon Intelligence facturent leurs accès entre 40 000 € et 150 000 € par an, excluant de facto les PME, ETI et organisations publiques régionales. 4. **La lourdeur des outils open source actuels** : Des projets respectables comme OpenCTI nécessitent des clusters d'infrastructure considérables (Elasticsearch, Redis, MinIO, RabbitMQ), demandant 16 Go de RAM minimum au démarrage, ce qui rend leur intégration impossible dans des environnements contraints ou des laboratoires d'ingénierie agiles.

2.2 Problématique d'Ingénierie et Besoins Utilisateurs

La problématique centrale à laquelle SENTRY répond est la suivante :

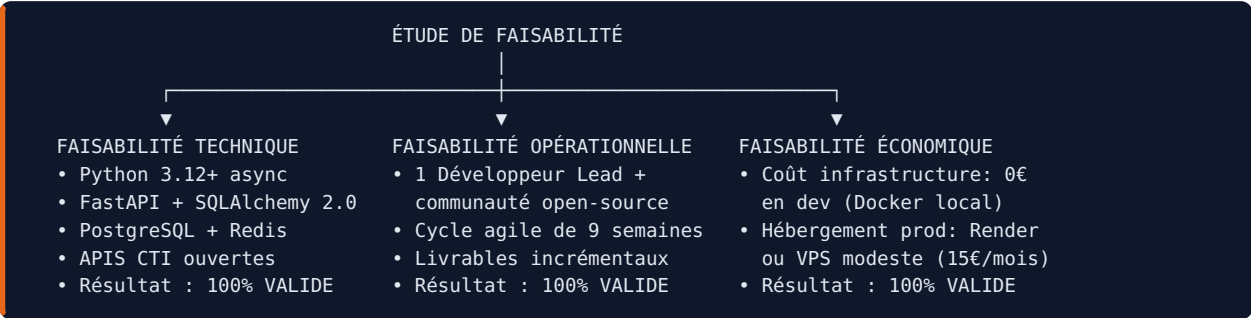
"Comment concevoir une plateforme de renseignement cyber robuste, légère et modulaire, capable d'ingérer de multiples flux hétérogènes en continu, de calculer un score de risque composite déterministe et d'orchestrer la réponse à incident, tout en s'exécutant avec une empreinte mémoire inférieure à 512 Mo et un déploiement clé en main ?"

Besoins Utilisateurs Identifiés :

- **BU-01 (Analyste SOC N1/N2)** : Disposer d'une source unique de vérité où chaque alerte est pré-enrichie avec son contexte géopolitique, technique et sa sévérité réelle.
 - **BU-02 (Threat Hunter / Analyste CTI)** : Pouvoir filtrer rapidement des dizaines de milliers d'indicateurs par famille d'attaque et exporter des listes de blocage (formats JSON/CSV/STIX).
 - **BU-03 (RSSI / Responsable Sécurité)** : Obtenir des métriques de pilotage consolidées (KPIs) pour mesurer la vitesse de traitement (MTTR, MTTD) et prioriser les budgets de remédiation technique.
 - **BU-04 (Ingénieur Système & DevOps)** : Avoir un binaire ou une image légère pilotable intégralement par API REST et par CLI, intégrable dans des pipelines CI/CD et des scripts d'automatisation.
-

2.3 Étude d'Opportunité et Analyse de Faisabilité

Une analyse triangulaire de faisabilité a été menée préalablement à la conception :



- Faisabilité Technique :** Les protocoles et API cibles (NVD REST 2.0, CISA KEV JSON, FIRST EPSS API, AlienVault OTX REST) sont publiquement documentés et stables. L'écosystème Python offre une maturité incomparable pour les requêtes HTTP asynchrones non-bloquantes (`httpx`), la validation des données (`pydantic`) et la persistance relationnelle performante (`asyncpg` / `SQLAlchemy`).
- Faisabilité Opérationnelle :** Le projet est calibré sur **9 semaines réparties en 6 phases modulaires**, représentant un investissement net de ~50 heures d'ingénierie pure. Chaque tâche est découpée en unité élémentaire de 20 à 45 minutes, garantissant une progression mesurable sans risque de blocage monolithique.
- Faisabilité Économique :** L'intégralité des briques logicielles sélectionnées est régie par des licences libres (MIT, BSD, Apache 2.0). Aucun coût de licence tiers n'est requis pour la v1.0. L'infrastructure de dev fonctionne en local sous Docker, et la cible de production cloud est dimensionnable sur des instances standard très économiques.

2.4 Matrice des Risques Pré-Projet et Stratégies d'Atténuation

ID Risque	Intitulé du Risque	Prob.	Impact	Gravité	Mesure Préventive & Stratégie d'Atténuation
RSK-01	Rate Limiting sévère des API tierces (ex: NVD NIST)	Élevée	Élevé	CRITIQUE	Implémenter un cache Redis local, supporter les clés API optionnelles, respecter un backoff exponentiel (<code>tenacity</code>) et espacer les requêtes.
RSK-02	Indisponibilité d'un flux externe	Élevée	Moyen	MAJEUR	Isoler chaque collecteur dans un worker indépendant ; l'échec d'un flux ne doit jamais faire planter les autres collecteurs ni l'API.
RSK-03	Explosion de la volumétrie des IOCs (mémoire/disque)	Moyenne	Élevé	MAJEUR	Mettre en place un cycle de vie strict des indicateurs (Aging/TTL) et une table avec indexation B-Tree et contraintes d'unicité pour déduplication immédiate.
RSK-04	Dérive du périmètre fonctionnel (Scope Creep)	Moyenne	Élevé	MAJEUR	Geler formellement le périmètre v1.0 sur les 44 tâches planifiées. Toute idée nouvelle est orientée vers le backlog v2.0 (Phase 7/8).
RSK-05	Régression de code lors des itérations rapides	Moyenne	Moyen	MODÉRÉ	Imposer une suite de tests unitaires et d'intégration (<code>pytest</code>) bloquant tout commit via GitHub Actions si le taux de succès n'est pas de 100%.

2.5 Analyse SWOAT du Projet SENTRY

Complément moderne du SWOT, l'analyse SWOAT évalue la viabilité stratégique en intégrant les aspirations propres à la vision CYBERILL :

• S - Forces (Strengths) :

- Architecture légère, asynchrone, modulaire et hautement performante.
- Scoring de risque composite exclusif combinant 4 dimensions (Sévérité, Exploitation, Prédiction, Attaque).
- Double interface native : API REST complète pour les intégrations + CLI riche pour les opérations rapides.
- Indépendance totale vis-à-vis des solutions fermées propriétaires américaines.

• W - Faiblesses (Weaknesses) :

- Projet jeune en phase de construction initiale.
- Dépendance vis-à-vis de la connectivité et de la structure des API publiques externes.
- Absence d'interface graphique lourde (IHM) en Phase 1 (choix assumé au profit de l'API et du CLI).

• O - Opportunités (Opportunities) :

- Entrée en vigueur des exigences réglementaires européennes NIS 2 imposant la surveillance CTI aux organisations.

- Forte demande du marché pour des briques CTI autonomes et transparentes pouvant tourner sur site (On-Premise).
 - Possibilité de fédérer une communauté internationale de chercheurs et d'ingénieurs autour du projet.
 - **A - Aspirations (Aspirations) :**
 - Positionner SENTRY et CyberillSec comme le standard open-source européen de surveillance des menaces d'ici 2030.
 - Démontrer un niveau d'excellence technique et d'architecture logicielle égal aux standards des plus grands centres de R&D.
 - **T - Menaces (Threats) :**
 - Évolution ou fermeture soudaine des points d'accès API gratuits par les fournisseurs de données.
 - Concurrence indirecte d'outils massifs pré-existants bien financés mais plus lourds.
-

3. CAHIER DES CHARGES FONCTIONNEL (CDCF)

Le présent chapitre décrit avec une précision chirurgicale l'ensemble des fonctionnalités que le système SENTRY doit offrir à ses utilisateurs. Chaque exigence est codifiée pour permettre la traçabilité complète lors des phases de développement et de test.

3.1 Vue d'ensemble des 6 Modules Métier

L'application SENTRY est scindée en 6 modules hautement cohésifs et faiblement couplés :

SENTRY CORE		
[MOD-01] Foundation Configuration, Base SQL, Authentification, CLI & Migrations Alembic	[MOD-02] Threat Feeds Collecte multi-sources, Normalisation, IOC Engine, Déduplication & Aging	[MOD-03] CVE Tracker API NVD 2.0, CISA KEV, Algorithme Risk Score, Moteur d'Alerting
[MOD-04] Incidents Modélisation cycle vie, Workflow 6 états, Timeline, Liens IOC/CVE	[MOD-05] SOC Dashboard Synthèse des métriques, Timeline d'activités, Export structuré JSON/CSV	[MOD-06] Threat Hunting Moteur de pattern-matching, Règles Sigma/KQL, Exécution de requêtes de chasse

3.2 Matrice des Exigences Fonctionnelles Globales

Réf. Exigence	Module Associé	Intitulé & Description Fonctionnelle	Priorité
RF-01	MOD-01	Le système doit charger sa configuration de manière immuable depuis l'environnement ou un fichier <code>.env</code> typé.	P0 (Critique)
RF-02	MOD-01	Le système doit initialiser la structure relationnelle de la base de données via des migrations de schéma automatisées.	P0 (Critique)
RF-03	MOD-01	Le système doit fournir une CLI permettant l'initialisation, le peuplement initial (seed) et le diagnostic de l'application.	P1 (Élevée)
RF-04	MOD-02	Le système doit permettre la création, consultation, modification et suppression (CRUD) de sources de flux de menaces (Threat Feeds).	P0 (Critique)
RF-05	MOD-02	Le système doit collecter des flux distants de façon asynchrone sans bloquer la boucle d'événements principale du serveur web.	P0 (Critique)
RF-06	MOD-02	Le système doit parser des flux sous formats structurés divers : JSON, CSV et STIX 2.1.	P0 (Critique)
RF-07	MOD-02	Le système doit extraire et stocker des Indicateurs de Compromission (IOCs) : IPs (v4/v6), Domaines, URLs, Hashes (MD5, SHA1, SHA256).	P0 (Critique)
RF-08	MOD-02	Le système doit dédupliquer automatiquement les IOCs : une même valeur observée plusieurs fois met à jour la date <code>last_seen</code> sans dupliquer la ligne.	P0 (Critique)
RF-09	MOD-02	Le système doit exécuter un worker périodique configurable pour actualiser automatiquement les flux actifs selon leur intervalle de rafraîchissement.	P1 (Élevée)
RF-10	MOD-02	Le système doit intégrer nativement le connecteur officiel AlienVault OTX via clé API.	P1 (Élevée)
RF-11	MOD-03	Le système doit se synchroniser avec l'API NIST NVD 2.0 pour importer les vulnérabilités CVE avec gestion de la pagination et rate-limit.	P0 (Critique)
RF-12	MOD-03	Le système doit importer le catalogue CISA KEV et marquer les CVE activement exploitées.	P0 (Critique)
RF-13	MOD-03	Le système doit importer les scores EPSS quotidiens de l'organisme FIRST pour évaluer la probabilité d'exploitation.	P0 (Critique)
RF-14	MOD-03	Le système doit calculer pour chaque CVE un Score de Risque Composite déterministe compris entre 0 et 100.	P0 (Critique)
RF-15	MOD-03	Le système doit offrir un moteur de recherche et filtrage multicritères sur les CVE (sévérité, date, statut KEV, seuil de score).	P1 (Élevée)
RF-16	MOD-03	Le système doit générer des alertes de sécurité immédiates dès lors qu'une vulnérabilité dépasse un seuil de risque prédéfini (ex: Score ≥ 75).	P1 (Élevée)
RF-17	MOD-04	Le système doit modéliser les incidents de sécurité et leur assigner une sévérité (Faible, Moyenne, Élevée, Critique).	P0 (Critique)

Réf. Exigence	Module Associé	Intitulé & Description Fonctionnelle	Priorité
RF-18	MOD-04	Le système doit implémenter une machine d'état stricte à 6 étapes pour le cycle de vie de l'incident (<i>Nouveau</i> → <i>Analyse</i> → <i>Confinement</i> → <i>Éradication</i> → <i>Récupération</i> → <i>Clôture</i>).	P0 (Critique)
RF-19	MOD-04	Le système doit enregistrer une chronologie immuable (Timeline d'événements) rattachée à chaque incident pour audit forensique.	P0 (Critique)
RF-20	MOD-04	Le système doit permettre d'associer des IOCs et des CVEs spécifiques à un incident donné.	P1 (Élevée)
RF-21	MOD-05	Le système doit exposer un point d'agrégation <code>/api/v1/dashboard/summary</code> compilant les compteurs et indicateurs clés en temps réel.	P0 (Critique)
RF-22	MOD-05	Le système doit retourner la liste consolidée des menaces et vulnérabilités les plus récentes des dernières 24 heures.	P1 (Élevée)
RF-23	MOD-05	Le système doit fournir une commande CLI <code>sentry dashboard show</code> affichant une vue console riche et colorée de l'état du SOC.	P1 (Élevée)
RF-24	MOD-05	Le système doit exporter les données du dashboard et des rapports aux formats standards structurés JSON et CSV.	P1 (Élevée)
RF-25	MOD-06	Le système doit intégrer un moteur d'évaluation de règles de détection par comparaison de motifs (Pattern Matching).	P1 (Élevée)
RF-26	MOD-06	Le système doit embarquer un catalogue initial de 5 règles de hunting opérationnelles représentatives des menaces réelles.	P1 (Élevée)
RF-27	MOD-06	Le système doit permettre l'exécution manuelle ou planifiée d'une session de hunting sur l'ensemble de la base d'IOCs.	P1 (Élevée)
RF-28	MOD-06	Le système doit enregistrer les résultats et correspondances d'une session de chasse avec le détail des entités détectées.	P1 (Élevée)

3.3 Spécification des Modules Métier

3.3.1 Module 1 : Foundation & Configuration

Ce module constitue le socle technique sur lequel reposent tous les autres. Il gère la configuration, la connexion au pool de base de données PostgreSQL, la gestion des migrations de schémas Alembic, et fournit l'interface CLI Click. * **Fonctions clés** : `sentry db init`, `sentry db upgrade`, `sentry version`, `sentry seed`. * **Règle de gestion** : Aucune requête applicative ne doit s'exécuter si la configuration n'est pas validée au démarrage par Pydantic.

3.3.2 Module 2 : Threat Feeds & Moteur IOC

Ce module orchestre l'alimentation en données brutes de renseignement. * **Format des IOCs supportés** : * `IPV4` : Validation par regex stricte et vérification d'exclusion des plages privées RFC 1918 (optionnelle). * `IPV6` : Validation conforme standard RFC 4291. * `DOMAIN` : Nom de domaine pleinement qualifié (FQDN). * `URL` : URI complète incluant schéma HTTP/HTTPS. * `HASH_MD5` : Hexadécimal 32 caractères. * `HASH_SHA1` : Hexadécimal 40 caractères. * `HASH_SHA256` : Hexadécimal 64 caractères. * **Algorithme de Déduplication** : Lors de l'ingestion d'un indicateur \$I = (\text{type}, \text{value})\$: \$\$\text{Si } I \in \text{Base} \implies \text{MAJ}(\text{last_seen} = \text{now}), \text{I}.

$\text{hit_count} = \text{hit_count} + 1$ \$\$\$
 $\text{Sinon } \implies \text{INSERT}(\text{first_seen} = \text{now}(), \text{last_seen} = \text{now}(), \text{hit_count} = 1)$ \$\$\$

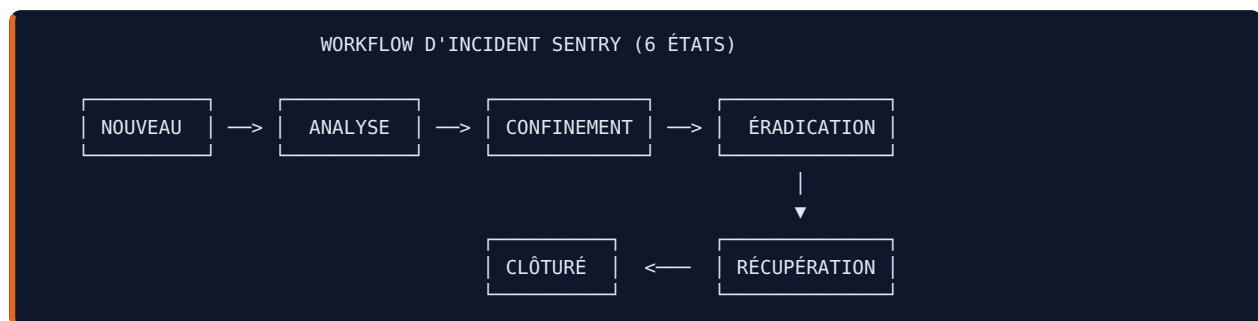
3.3.3 Module 3 : CVE Tracker & Vulnerability Intelligence

Ce module transforme les données publiques de vulnérabilités en aide à la décision priorisée. *

Formule Mathématique du Risk Score Composite (\$R\$) : Pour chaque CVE, le score \$R \in [0, 100]\$ est calculé ainsi : $R = \min(100, (\text{CVSS} \times 3.0) + (\text{EPSS} \times 100 \times 0.25) + (K \times 25) + (E \times 10) + (A \times 10))$ Où : * \$\text{CVSS} \in [0.0, 10.0]\$ est la sévérité technique brute. * \$\text{EPSS} \in [0.0, 1.0]\$ est la probabilité d'exploitation réelle. * \$K \in \{0, 1\}\$ indique la présence dans le catalogue CISA KEV (\$1\$ si présent, \$0\$ sinon). * \$E \in \{0, 1\}\$ indique l'existence d'un exploit public documenté (Exploit-DB / PoC). * \$A \in \{0, 1\}\$ indique une attaque ou campagne de ransomware confirmée. * **Grille de Décision SOC :** * \$R \geq 80\$: **CRITIQUE (P0)** → Alerte immédiate, patch sous 24 heures. * \$60 \leq R < 80\$: **ÉLEVÉ (P1)** → Alerte de quart, patch sous 7 jours. * \$40 \leq R < 60\$: **MOYEN (P2)** → Revue hebdomadaire, patch sous 30 jours. * \$R < 40\$: **FAIBLE (P3)** → Maintenance standard.

3.3.4 Module 4 : Incident Management & Workflow NIST SP 800-61

Gère le cycle de vie opérationnel d'un incident de sécurité selon la machine d'état officielle :



• Règles de transition d'état :

- Un incident ne peut être clôturé sans qu'une note de post-mortem ou un résumé de clôture soit renseigné.
- Chaque transition d'état génère automatiquement un événement immuable dans la table `incident_events`.

3.3.5 Module 5 : SOC Dashboard & Analytics

Fournit la synthèse exécutive et tactique de la plateforme : * Métriques en temps réel : nombre total d'IOCs actifs, ratio de sévérité des CVEs, nombre d'incidents ouverts par criticité, santé des collecteurs de flux. * Moteur d'export sérialisant les données en JSON structuré conforme RFC 8259 ou en CSV RFC 4180.

3.3.6 Module 6 : Threat Hunting Engine

Permet l'évaluation de règles de corrélation avancées sur les indicateurs collectés. * **Catalogue des 5**

Règles Minimales v1.0 : * **RULE-01 (Tor Exit Node Check)** : Détection de connexions réseau ou d'IOCs correspondant à des relais de sortie Tor connus. * **RULE-02 (Suspicious Dynamic DNS)** : Détection de domaines résolus chez des fournisseurs de DNS dynamique gratuits (ex: `duckdns.org`, `no-ip.com`) souvent utilisés par les malwares. * **RULE-03 (High-Entropy Domains / DGA)** : Détection de noms de domaine générés algorithmiquement (Domain Generation Algorithm) présentant une entropie de Shannon élevée. * **RULE-04 (Known Ransomware Extortion IP)** : Détection d'adresses IP associées aux infrastructures de fuite de données de ransomware (DLS). * **RULE-05 (Critical CVE + Exploit Match)** : Corrélation immédiate entre un actif exposé et une vulnérabilité ayant un exploit public vérifié.

3.9 Cas d'Usage Détaillés (Use Cases)

Cas d'Usage UC-01 : Ingestion Automatisée d'un Flux Threat Intelligence

ACTEUR PRIMAIRE : Système / Worker Périodique (Background Scheduler)
PRÉCONDITIONS : Un flux ThreatFeed est configuré et marqué "is_active = True".
DÉCLENCHEUR : L'intervalle de rafraîchissement (polling_interval) est écoulé.

SCÉNARIO NOMINAL :

1. Le worker interroge la table 'threat_feeds' et sélectionne les flux éligibles.
2. Le connecteur réseau asynchrone émet une requête GET vers l'URL du flux avec timeout de 15s.
3. Le serveur distant répond avec un code HTTP 200 et un payload (JSON ou STIX).
4. Le parseur spécifique valide le format et extrait la liste des indicateurs bruts.
5. Pour chaque indicateur, le sous-système de déduplication vérifie son existence en base.
6. Les nouveaux indicateurs sont insérés ; les indicateurs existants sont mis à jour (last_seen, hit_count).
7. Le flux 'threat_feed' est mis à jour avec : last_successful_run = now(), status = "HEALTHY".
8. Un événement de log structuré est émis avec le nombre d'indicateurs traités.

SCÉNARIOS ALTERNATIFS :

- 2a. Erreur réseau ou timeout (serveur distant injoignable) :
 - Le système effectue jusqu'à 3 tentatives avec backoff exponentiel.
 - Si l'échec persiste, le flux est marqué status = "DEGRADED", l'erreur est tracée dans les logs.
- 3a. Réponse HTTP 429 (Too Many Requests / Rate Limit) :
 - Le worker extrait l'en-tête 'Retry-After' ou applique une temporisation par défaut de 60s.
 - La collecte pour ce flux spécifique est reportée au cycle suivant.

4. CAHIER DES CHARGES TECHNIQUE (CDCT) & EXIGENCES NON-FONCTIONNELLES

4.1 Matrice des Exigences Non-Fonctionnelles (RNF)

Identifiant	Catégorie	Spécification Technique & Métrique d'Acceptation
RNF-SEC-01	Sécurité	Toutes les communications externes et API sont chiffrées obligatoirement en TLS 1.3 (ou TLS 1.2 strict). Aucun mot de passe ou secret en clair dans le code.
RNF-SEC-02	Sécurité	Les entrées utilisateur et paramètres d'API sont strictement sanitisés et typés via Pydantic pour éliminer tout risque d'injection SQL ou XSS.
RNF-PERF-01	Performance	Le temps de réponse (latence) au 95ème percentile (\$P_{95}\$) des requêtes de lecture de l'API (<code>/api/v1/cves</code> , <code>/api/v1/feeds</code>) doit être $< 250\text{ ms}$.
RNF-PERF-02	Performance	Le traitement et l'ingestion d'un lot de 1 000 IOCs en base ne doit pas excéder 5 secondes via des opérations d'insertion groupée (<code>bulk_insert / upsert</code>).
RNF-DISP-01	Disponibilité	Le serveur API doit assurer un taux de disponibilité minimal de 99.5% hors périodes de maintenance programmées.
RNF-MAIN-01	Maintenabilité	Le code source doit obtenir un score de conformité de 100% au linter <code>ruff</code> et une couverture de tests automatisés <code>pytest</code> $\geq 80\%$.
RNF-MEM-01	Ressources	L'empreinte mémoire totale de l'application FastAPI en régime nominal ne doit pas dépasser 256 Mo de RAM.

4.2 Justification Rigoureuse de la Stack Technologique

STACK SENTRY v1.0	
Langage Principal	Python 3.12+ (Asyncio natif, Typage strict)
Framework API Web	FastAPI v0.115+ (ASGI, OpenAPI auto, Rapide)
Moteur ORM & Base	SQLAlchemy v2.0 Async + PostgreSQL 16
Pilote Base de Données	asyncpg (Driver asynchrone le plus rapide)
Gestionnaire de Migrations	Alembic v1.13+
Cache & Courtier Tâches	Redis v7.2+ (Clé-valeur mémoire)
Validation des Schémas	Pydantic v2.7+ (Validation compilée en Rust)
Client HTTP Asynchrone	httpx v0.27+ (Support HTTP/2, SOCKS proxy)
Interface en Ligne de Cde	Click v8.1+ & Rich v13.7+ (Rendu console pro)
Suite de Tests	Pytest v8.0+, pytest-asyncio, pytest-cov
Conteneurisation	Docker v26+ & Docker Compose v2

Pourquoi ce choix plutôt qu'une autre stack (Node.js ou Go) ?

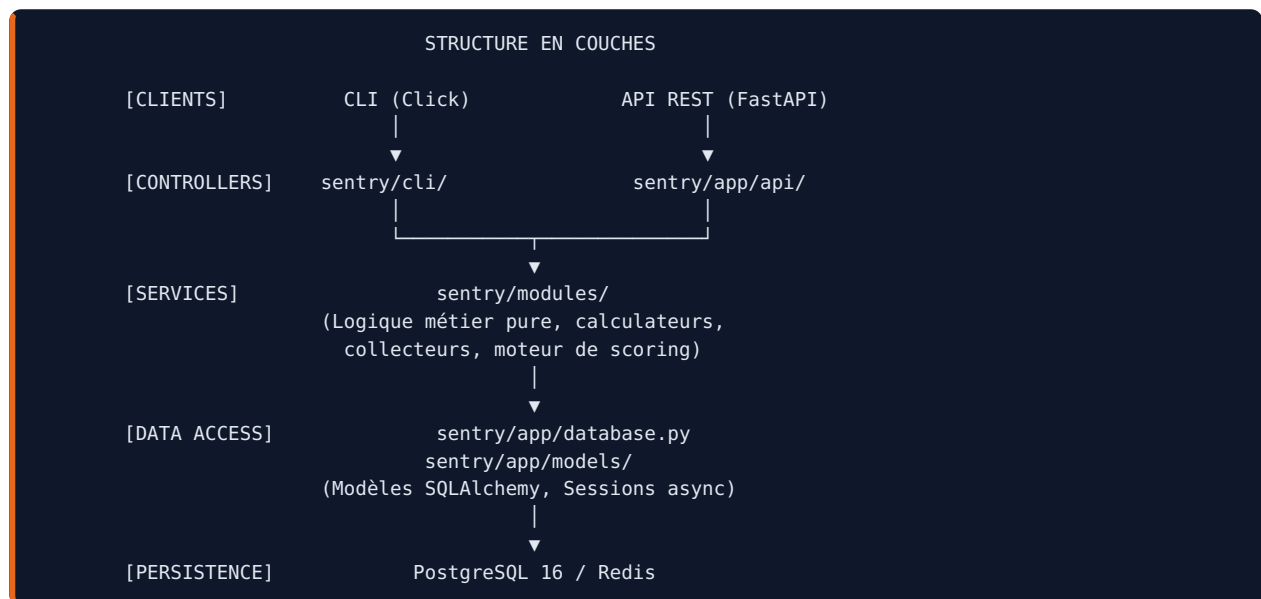
1. **L'écosystème CTI :** L'intégralité des standards mondiaux en cybersécurité (outils d'analyse, STIX, TAXII, bindings YARA, parsers Sigma) sont développés prioritairement en Python.

2. **FastAPI & Pydantic v2 :** En utilisant le

moteur de Pydantic v2 réécrit en Rust, FastAPI offre des débits de requêtes proches de Go tout en conservant l'expressivité et la simplicité de Python. 3. **SQLAlchemy 2.0 Async** : Permet une abstraction relationnelle complète sans bloquer les threads du serveur lors des accès disques ou réseau.

4.3 Architecture Logicielle : Clean Architecture Adaptée

Le code de SENTRY est rigoureusement structuré en couches selon le principe de séparation des responsabilités (Separation of Concerns) :



- **Règle d'or pour le développeur** : Un contrôleur d'API ne fait **JAMAIS** de calcul métier ni de requête SQL complexe directe. Il valide les paramètres d'entrée, appelle la couche **Service**, et retourne le schéma Pydantic de réponse.

4.4 Modèle Relationnel de Données (Schémas DDL)

Voici les structures exactes des tables relationnelles qui composent la base de données SENTRY :

```

-- 1. Table des Utilisateurs et Analystes
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  hashed_password VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL DEFAULT 'ANALYST', -- ADMIN, ANALYST, VIEWER
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Table des Flux de Renseignement (Threat Feeds)
CREATE TABLE threat_feeds (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) UNIQUE NOT NULL,
  url VARCHAR(500) NOT NULL,
  feed_type VARCHAR(20) NOT NULL, -- STIX, JSON, CSV
  polling_interval INTEGER NOT NULL DEFAULT 3600, -- en secondes
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  last_successful_run TIMESTAMP WITH TIME ZONE,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING', -- PENDING, HEALTHY, DEGRADED
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Table des Indicateurs de Compromission (IOCs)
CREATE TABLE indicators (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  feed_id UUID REFERENCES threat_feeds(id) ON DELETE SET NULL,
  type VARCHAR(20) NOT NULL, -- IPV4, DOMAIN, URL, HASH_SHA256, etc.
  value TEXT NOT NULL,
  severity VARCHAR(20) NOT NULL DEFAULT 'MEDIUM', -- LOW, MEDIUM, HIGH, CRITICAL
  description TEXT,
  hit_count INTEGER NOT NULL DEFAULT 1,
  first_seen TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  last_seen TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_indicator_type_value UNIQUE (type, value)
);
CREATE INDEX idx_indicators_type_val ON indicators (type, value);
CREATE INDEX idx_indicators_last_seen ON indicators (last_seen DESC);

-- 4. Table des Vulnérabilités (CVE Tracker)
CREATE TABLE cves (
  id VARCHAR(30) PRIMARY KEY, -- ex: CVE-2026-16812
  description TEXT NOT NULL,
  cvss_score NUMERIC(3, 1), -- ex: 9.8
  epss_score NUMERIC(5, 4), -- ex: 0.8920
  is_kev BOOLEAN NOT NULL DEFAULT FALSE,
  has_public_exploit BOOLEAN NOT NULL DEFAULT FALSE,
  composite_risk_score NUMERIC(5, 2) NOT NULL DEFAULT 0.0, -- 0 à 100
  published_date TIMESTAMP WITH TIME ZONE NOT NULL,
  last_modified_date TIMESTAMP WITH TIME ZONE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_cves_risk_score ON cves (composite_risk_score DESC);

-- 5. Table des Incidents de Sécurité
CREATE TABLE incidents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(200) NOT NULL,
  description TEXT NOT NULL,
  severity VARCHAR(20) NOT NULL, -- LOW, MEDIUM, HIGH, CRITICAL
  status VARCHAR(30) NOT NULL DEFAULT 'NOUVEAU', -- NOUVEAU, ANALYSE, CONFINEMENT, ERADICATION, RECUPERATION, CLOTURE
  assigned_to UUID REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  closed_at TIMESTAMP WITH TIME ZONE

```

```
);

-- 6. Table de la Chronologie des Incidents (Timeline Events)
CREATE TABLE incident_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id UUID NOT NULL REFERENCES incidents(id) ON DELETE CASCADE,
  author_id UUID REFERENCES users(id) ON DELETE SET NULL,
  event_type VARCHAR(50) NOT NULL, -- STATUS_CHANGE, COMMENT, IOC_ATTACHED, ACTION_TAKEN
  message TEXT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_incident_events_timeline ON incident_events (incident_id, created_at ASC);
```

4.5 Spécifications des Contrats d'Interface API REST

Tous les endpoints sont préfixés par `/api/v1`. Les requêtes et réponses échangent exclusivement des charges utiles au format `application/json`.

Synthèse des Endpoints Principaux :

Méthode	Route	Description	Code Succès
GET	/health	Diagnostic de vivacité et vérification de la connexion DB	200 OK
GET	/api/v1/feeds	Liste paginée de tous les flux de menaces configurés	200 OK
POST	/api/v1/feeds	Enregistrement d'un nouveau flux source	201 Created
POST	/api/v1/feeds/{id}/fetch	Déclenchement manuel immédiat de la collecte sur un flux	202 Accepted
GET	/api/v1/indicators	Recherche filtrée sur les IOCs (type, valeur, sévérité)	200 OK
GET	/api/v1/cves	Consultation des CVEs avec tri par score de risque	200 OK
GET	/api/v1/cves/critical	Extraction directe des CVEs prioritaires (\$R \ge 75\$)	200 OK
GET	/api/v1/incidents	Liste des incidents actifs avec état et assignation	200 OK
POST	/api/v1/incidents	Déclaration d'un nouvel incident de sécurité	201 Created
PATCH	/api/v1/incidents/{id}/status	Transition de l'incident vers un nouvel état	200 OK
GET	/api/v1/dashboard/summary	Indicateurs consolidés pour le tableau de bord SOC	200 OK
POST	/api/v1/hunt/run	Exécution d'une session de hunting sur les indicateurs	200 OK

5. PLANNING OPÉRATIONNEL & DÉCOUPAGE DES 44 TÂCHES

Le projet SENTRY se déploie sur **9 semaines réparties en 6 phases séquentielles**, avec un lancement officiel à J1 (30/07/2026) et une clôture de la v1.0 à J63 (30/09/2026).

5.1 Macro-Planning et Calendrier des Phases

Période globale : 30 juillet 2026 —> 30 septembre 2026 (63 jours / 9 semaines)

PHASE 1 (J01-J07) : Foundation & Squelette Applicatif	[30/07 - 06/08]	(8h)
PHASE 2 (J08-J21) : Threat Feeds & Moteur IOC	[06/08 - 20/08]	(12h)
PHASE 3 (J22-J35) : CVE Tracker & Surveillance Vuln.	[20/08 - 03/09]	(10h)
PHASE 4 (J36-J49) : Incident Management & Workflow	[03/09 - 17/09]	(10h)
PHASE 5 (J50-J56) : SOC Dashboard & Analytics	[17/09 - 24/09]	(6h)
PHASE 6 (J57-J63) : Threat Hunting & Clôture v1.0	[24/09 - 01/10]	(5h)

5.2 Les 6 Jalons Majeurs d'Acceptation (Milestones)

- **[JALON M1] — 05/08/2026 : Squelette Opérationnel Validé**
 - *Critères de validation* : Le serveur FastAPI démarre sous Docker, la base PostgreSQL se migre via Alembic sans erreur, l'endpoint `/health` retourne `200 OK` avec statut DB connecté, et la suite de tests unitaires passe à 100%.
- **[JALON M2] — 19/08/2026 : Ingestion Threat Feeds Opérationnelle**
 - *Critères de validation* : Collecteur AlienVault OTX et flux STIX connectés, déduplication fonctionnelle, et un stock minimum de 500 indicateurs réels persisté en base de données.
- **[JALON M3] — 02/09/2026 : Moteur CVE & Alerting Opérationnel**
 - *Critères de validation* : API NVD 2.0 synchronisée, scores EPSS intégrés, algorithme de Risk Score calculé et vérifié par tests unitaires, système d'alerte déclenché sur CVE critique.
- **[JALON M4] — 16/09/2026 : Gestion des Incidents Opérationnelle**
 - *Critères de validation* : Workflow d'incident testé sur les 6 états nominaux, timeline d'événements persistée, liaison bidirectionnelle d'incidents avec des IOCs et CVEs validée.
- **[JALON M5] — 23/09/2026 : Dashboard SOC & Reporting Opérationnels**
 - *Critères de validation* : Endpoint `/dashboard/summary` fonctionnel avec métriques exactes, commande `sentry dashboard show` opérationnelle en console, exports JSON/CSV conformes.
- **[JALON FINAL] — 30/09/2026 : SENTRY v1.0 Validée & Prête pour la Production**
 - *Critères de validation* : Les 6 modules intégrés, couverture de code globale $\geq 80\%$, documentation complète à jour et tag Git `v1.0.0` poussé sur GitHub et GitLab.

5.3 Décomposition Détaillée des 44 Activités Quotidiennes

Chaque tâche est un incrément précis permettant au développeur de livrer une valeur testable :

Phase 1 : Foundation (J1-J7)

1. **30/07/2026** — **T1.1** : Mise en place de l'arborescence standard, configuration de `pyproject.toml`, dépendances de production et squelette de l'application FastAPI.
2. **30/07/2026** — **T1.2** : Configuration du module de base de données `database.py`, déclaration du modèle `User` et génération de la migration initiale Alembic.
3. **31/07/2026** — **T1.3** : Implémentation du CLI Click racine (`sentry`) avec sous-commandes de diagnostic (`db init` , `db upgrade` , `version`).
4. **31/07/2026** — **T1.4** : Rédaction du fichier `docker-compose.yml` orchestrant les conteneurs PostgreSQL 16 et Redis 7 avec volumes persistants.
5. **01/08/2026** — **T1.5-1.7** : Déclaration et isolation des répertoires modules applicatifs (`threat_feeds` , `cve_tracker` , `incidents`) avec leurs fichiers `__init__.py` .
6. **01/08/2026** — **T1.8** : Mise en place du framework de tests `pytest` , configuration des fixtures asynchrones et validation du premier test unitaire sur `/health` .

Phase 2 : Threat Feeds (J8-J21)

1. **06/08/2026** — **T2.1** : Conception du modèle SQLAlchemy `ThreatFeed` et application de la migration Alembic associée.
2. **06/08/2026** — **T2.2** : Développement des contrôleurs API CRUD pour la route `/api/v1/feeds` avec schémas Pydantic de validation.
3. **07/08/2026** — **Activité Externe** : Création du compte de recherche AlienVault OTX et récupération de la clé API pour l'environnement.
4. **07/08/2026** — **T2.3** : Développement du client HTTP asynchrone générique avec parseurs de flux aux formats JSON, CSV et STIX.
5. **08/08/2026** — **T2.4** : Conception du modèle relationnel `Indicator` (table `indicators`) et application de la migration.
6. **08/08/2026** — **T2.5** : Implémentation du mécanisme d'insertion optimisée et déduplication automatique des indicateurs (upsert).
7. **09/08/2026** — **T2.6** : Développement du composant d'arrière-plan (Background Task Worker) ordonnant les collectes automatiques.
8. **10/08/2026** — **T2.7** : Ajout des commandes d'administration CLI pour les flux (`sentry feeds list` , `add` , `fetch` , `fetch-all`).
9. **10/08/2026** — **T2.8** : Écriture de la suite de tests unitaires et de mocks réseau pour le module `threat_feeds` .
10. **15/08/2026** — **T2.9** : Développement du connecteur dédié à l'API AlienVault OTX et extraction des pulses récents.
11. **16/08/2026** — **T2.10** : Intégration et validation d'un parseur de paquets de renseignement au standard STIX/TAXII.

Phase 3 : CVE Tracker (J22-J35)

1. **20/08/2026** — **T3.1** : Modélisation de la table `cves` , migration Alembic et injection du token API NIST NVD.
2. **21/08/2026** — **T3.2** : Développement du collecteur pour l'API REST v2.0 du NIST NVD avec gestion fine du débit de requêtes.
3. **22/08/2026** — **T3.3** : Pipeline de normalisation des champs de vulnérabilité (descriptions, vecteurs CVSS, métriques).
4. **23/08/2026** — **T3.4** : Implémentation du moteur de recherche et filtrage multi-critères en base de données.

5. **24/08/2026** — **T3.5** : Exposition des endpoints REST `/api/v1/cves` et `/api/v1/cves/{cve_id}`.
6. **25/08/2026** — **T3.6** : Implémentation de la fonction mathématique de Risk Score Composite et du moteur de déclenchement d'alertes.
7. **26/08/2026** — **T3.7** : Développement des commandes CLI associées (`sentry cves list`, `search`, `alert`).
8. **27/08/2026** — **T3.8** : Couverture de tests automatisés vérifiant la précision des calculs de scoring et la tolérance aux pannes réseau.

Phase 4 : Incident Management (J36-J49)

1. **03/09/2026** — **T4.1** : Création du modèle relationnel `Incident` et exécution de la migration de schéma.
2. **04/09/2026** — **T4.2** : Conception de la table `incident_events` matérialisant la timeline chronologique.
3. **05/09/2026** — **T4.3** : Codage de la machine d'état régissant les 6 étapes de vie de l'incident avec garde-fous de transition.
4. **06/09/2026** — **T4.4** : Développement des contrôleurs REST `/api/v1/incidents` (création, consultation, mise à jour).
5. **07/09/2026** — **T4.5** : Implémentation de l'endpoint dédié à l'enrichissement et la lecture de la timeline d'incident.
6. **08/09/2026** — **T4.6** : Développement des tables de liaison d'incidents avec les IOCs suspects et les CVEs exploitées.
7. **09/09/2026** — **T4.7** : Implémentation des fonctionnalités CLI d'incidentologie (`sentry incidents list`, `create`, `close`).
8. **10/09/2026** — **T4.8** : Rédaction des tests unitaires validant l'intégrité de la machine d'état et des transitions interdites.

Phase 5 : SOC Dashboard (J50-J56)

1. **17/09/2026** — **T5.1** : Développement du service d'agrégation et de l'endpoint `/api/v1/dashboard/summary`.
2. **18/09/2026** — **T5.2** : Implémentation du contrôleur d'extraction des menaces récentes des dernières 24 heures.
3. **19/09/2026** — **T5.3** : Développement de l'endpoint d'historique consolidé des événements sur 7 jours glissants.
4. **20/09/2026** — **T5.4** : Conception du tableau de bord console dynamique via la bibliothèque Python `rich` (`sentry dashboard show`).
5. **21/09/2026** — **T5.5** : Moteur de sérialisation et d'export des données compilées sous formats JSON et CSV.
6. **22/09/2026** — **T5.6** : Validation complète par tests d'intégration des métriques et des formats d'export.

Phase 6 : Threat Hunting (J57-J63)

1. **24/09/2026** — **T6.1** : Implémentation du moteur d'évaluation de règles de corrélation par pattern matching.
2. **25/09/2026** — **T6.2** : Codage et intégration en dur des 5 règles opérationnelles de chasse (Tor, DynDNS, DGA, Ransomware, Exploit Match).
3. **26/09/2026** — **T6.3** : Développement des routes API `/api/v1/hunt/rules` et `/api/v1/hunt/run`.

4. **27/09/2026** — **T6.4** : Ajout des commandes CLI de hunting (`sentry hunt run` , `list-rules` , `results`).
 5. **28/09/2026** — **T6.5** : Suite de tests finaux validant l'efficacité des détections et la précision des correspondances.
-

5.4 Protocole des Bilans Hebdomadaires du Jeudi

Chaque jeudi soir, sans exception, l'équipe d'ingénierie compile l'avancement de la semaine écoulée dans un rapport d'audit normé situé sous `Rapport/NN_date.md` et poussé sur le dépôt GitHub.

Structure obligatoire du Rapport Hebdomadaire :

```
# SENTRY – Rapport d'Avancement Hebdomadaire [Semaine NN/9]
**Date :** [JJ/MM/AAAA] | **Auteur :** [Nom du Développeur] | **Jalon Cible :** [M1 à M5]

## 1. Résumé Exécutif
Synthèse en 5 lignes des réalisations de la semaine pour le management et le RSSI.

## 2. Détail Technique des Tâches Traitées
- [x] Tâche X.X : Description technique, commits associés, fichiers créés/modifiés.
- [x] Tâche X.X : Tests rédigés et résultats d'exécution.

## 3. Analyse SWOT de la Semaine
- **Forces :** Composants stables et performants livrés.
- **Faiblesses :** Dette technique identifiée ou documentation incomplète.
- **Opportunités :** Optimisations possibles découvertes lors du développement.
- **Menaces :** Blocages externes ou retards potentiels.

## 4. Problèmes Rencontrés & Solutions Apportées
Tableau récapitulant chaque blocage technique (ex: rate-limit API, bug asyncpg) et la solution adoptée.

## 5. Métriques Qualité & Indicateurs d'Ingénierie
- Taux de couverture de test pytest : XX%
- Nombre de commits réalisés : XX
- Statut de conformité du linter Ruff : 100% OK
```

6. GUIDE D'ONBOARDING & MANUEL DU DÉVELOPPEUR (DAY-1)

Ce guide pratique te prend par la main pour configurer ton poste de travail et livrer ta première fonctionnalité en moins d'une heure.

6.1 Prérequis Système et Installation Pas à Pas

Assure-toi d'avoir installé sur ta machine : * Un environnement Linux (Ubuntu 22.04+ ou Debian 12 recommandé), macOS ou WSL2 sous Windows. * **Python 3.12** ou supérieur (`python3 --version`). * **Git** (`git --version`). * **Docker** et **Docker Compose** (`docker compose version`).

Procédure d'installation initiale :

```
# 1. Cloner le dépôt de code officiel
git clone https://github.com/GodwillFoka/sentry.git
cd sentry

# 2. Créer l'environnement virtuel Python isolé
python3 -m venv venv
source venv/bin/activate

# 3. Mettre à jour pip et installer les dépendances de développement
pip install --upgrade pip
pip install -r requirements.txt
pip install pytest pytest-asyncio pytest-cov ruff mypy

# 4. Initialiser la configuration d'environnement locale
cp .env.example .env

# 5. Démarrer les services d'infrastructure (PostgreSQL & Redis)
docker compose up -d

# 6. Exécuter les migrations de base de données
alembic upgrade head

# 7. Vérifier que tout fonctionne en lançant la suite de tests
pytest -v

# 8. Lancer le serveur d'API en mode rechargement automatique (Hot-Reload)
uvicorn sentry.app.main:app --reload --host 0.0.0.0 --port 8000
```

Ouvre ton navigateur sur `http://localhost:8000/docs` : la documentation interactive Swagger doit s'afficher immédiatement.

6.2 Structure Physique du Dépôt Expliquée

Voici l'organisation exacte des répertoires du projet que tu vas manipuler :

```

sentry/
├── .github/workflows/      # Définition des pipelines CI/CD GitHub Actions
├── alembic/                # Scripts de migration de schéma de base de données
│   └── versions/          # Historique immuable des migrations SQL
├── sentry/                 # Code source applicatif principal
│   ├── app/               # Noyau de l'API web
│   │   ├── api/           # Contrôleurs et définitions des routes REST
│   │   ├── models/        # Modèles de données relationnels SQLAlchemy
│   │   ├── config.py       # Définition Pydantic des variables d'environnement
│   │   ├── database.py     # Moteur et sessions asynchrones de base de données
│   │   └── main.py         # Point d'entrée de l'application FastAPI
│   ├── cli/               # Commandes en ligne de commande Click (sentry ...)
│   ├── modules/           # Modules métier indépendants (Clean Architecture)
│   │   ├── threat_feeds/   # Moteur de collecte, parseurs STIX/JSON et IOCs
│   │   ├── cve_tracker/    # Connecteur NVD, EPSS et Risk Scoring
│   │   ├── incidents/      # Machine d'état et gestion des incidents
│   │   ├── dashboard/      # Agrégateur analytique et exports
│   │   └── threat_hunting/  # Moteur de détection et règles opérationnelles
│   └── shared/             # Utilitaires transverses, helpers et constantes
├── tests/                 # Suite de tests automatisés Pytest
│   ├── conftest.py         # Fixtures asynchrones et base de test éphémère
│   ├── test_api.py         # Tests d'intégration des endpoints web
│   └── test_models.py       # Tests unitaires des règles métier et du scoring
├── .env.example            # Modèle des variables d'environnement documentées
├── docker-compose.yml      # Définition des services PostgreSQL et Redis locaux
├── pyproject.toml          # Configuration des outils Python (Ruff, Mypy, Pytest)
└── requirements.txt        # Liste verrouillée des dépendances applicatives

```

6.3 Standards d'Ingénierie & Règles de Qualité

Tout code soumis doit respecter sans dérogation les 4 piliers de qualité de CYBERILL :

1. **Typage Strict** : Toutes les signatures de fonctions doivent être typées statiquement. `mypy` est exécuté en mode strict. ``python # BON : async def calculate_risk(cve_id: str, cvss: float, is_kev: bool) -> float: ...

INTERDIT : def calculate_risk(cve_id, cvss, is_kev): ... `` 2. ****Linting & Formatage**** : Avant de commiter, exécute `ruff check` et `ruff format`. Aucune ligne ne doit dépasser 100 caractères. 3. ****Tests Obligatoires**** : Toute nouvelle route d'API ou fonction de calcul doit être accompagnée de son test correspondant dans le dossier `tests/`. Le taux de succès doit être de 100%. 4. ****Gestion des Exceptions**** : Ne capture jamais d'exception générique silencieusement (`except: pass`). Chaque exception doit être tracée de manière explicite et typée.

6.4 GitFlow, Conventional Commits et Workflow de Contribution

• Stratégie de Branches :

- La branche `main` est sacrée. Aucun commit direct n'est autorisé sur `main`.
- Pour chaque tâche, crée une branche dédiée issue de `main` : `feat/T2.1-threat-feed-model` ou `fix/T3.3-cve-parser-bug`.

- **Format des Messages de Commit (Conventional Commits)** : Chaque commit doit obligatoirement respecter le format normé :

- `feat(feeds): add STIX 2.1 parser for alienvault otx`
- `test(cve): add unit tests for composite risk scoring formula`
- `fix(database): resolve asyncpg connection pool timeout`
- `docs(onboarding): update day-1 guide for docker compose setup`

6.5 Procédure Pas à Pas pour Réaliser sa Première Tâche

Voici la routine exacte que tu dois appliquer chaque jour :

1. **Synchronisation** : Mets à jour ta branche locale `git checkout main && git pull origin main`.
2. **Création de Branche** : Crée ta branche de travail : `git checkout -b feat/T3.6-cve-alerting`.
3. **Développement** : Rédige le code de ta fonctionnalité dans le sous-dossier module approprié.
4. **Tests en Local** : Rédige le test associé dans `tests/` et exécute `pytest -v`.
5. **Vérification du Style** : Exécute `ruff check .` pour vérifier qu'aucune alerte de style ne subsiste.
6. **Commit & Push** : Effectue ton commit propre et pousse ta branche : `bash git add . git commit -m "feat(cve): implement threshold-based alerting logic" git push origin feat/T3.6-cve-alerting`
7. **Pull Request** : Ouvre une Pull Request sur GitHub vers la branche `main`. Renseigne la description avec les critères de validation testés. Dès que la CI est verte et validée, la fusion est effectuée.

FIN DU CAHIER DES CHARGES SENTRY
Document de Référence pour l'Ingénierie Logicielle CYBERILL
"Engineering Cyber Resilience. Empowering Digital Trust."
